Hybrid RAG Interview Preparation

Enterprise Multi-Source Retrieval: Architecture, Evaluation, and Operations

Parth Jaiswal | September 2026

1.92M **CHUNKS** across 9 enterprise data sources	**34M** **ENTITY ROWS** knowledge graph in one SQLite file	**16.9 GB** **DATABASE** single file, FTS5 index included	**500** **EVAL QUESTIONS** 10 question types, both pipelines
-7.7% **TOKEN USAGE** 459,014 vs 423,874 total tokens	**+3 pp** **WIN RATE** 36% to 39% vs Standard RAG	**+0.9 pp** **RELEVANCE** 65.3% to 66.1% answer relevance	**~10 s** **LATENCY COST** Hybrid vs Standard, per query

What this document is

A complete preparation guide for interviews about the Hybrid RAG system: how the pipeline works stage by stage (with a real worked example), what every technique means (BM25, FTS5, RRF, cross-encoders and more), how the database is designed, how memory and state are handled, what failed and how it was fixed, where the scaling walls are, why each tool was chosen, and model answers to the five core agent-system interview questions. All numbers come from the 500-question evaluation or from live container tests unless marked otherwise.

1. The System in Sixty Seconds

The system is an enterprise search engine over nine workplace data sources (Slack, Gmail, GitHub, Jira, Confluence, HubSpot, Linear, Fireflies meeting transcripts, Google Drive). Nine ingestion pipelines chunk each source's documents into 1,924,473 chunks and extract named entities into a 34-million-row knowledge graph, both stored in a single 16.9 GB SQLite database with a built-in FTS5 full-text index. At query time a smart router decides between a cheap lexical pipeline (Standard RAG) and an entity-aware hybrid pipeline (Hybrid RAG) that fuses BM25 search with entity-graph traversal using Reciprocal Rank Fusion. The winning chunks become cited context for an LLM answer. On a 500-question evaluation, the hybrid path cut token usage by 7.7% and raised answer relevance by 0.9 percentage points, at roughly 10 seconds of extra latency per query.

Component choices

Component	Choice	Why
Storage + lexical index	SQLite with FTS5 extension	One zero-config file; BM25 full-text search built in; ships inside the Docker image
Entity extraction	spaCy en_core_web_sm NER	Deterministic, runs on CPU in milliseconds, no per-query LLM cost
Retrieval fusion	Reciprocal Rank Fusion (RRF)	Combines BM25 ranking and entity-graph ranking without tuning score scales
Reranking	MiniLM cross-encoder (tier 4, last resort only)	Accurate but costly; only invoked when retrieval confidence is low
Answer generation	OpenAI GPT (user-supplied API key)	Strong instruction following for citation-only answers; users pay their own cost
Web UI	Gradio (port 7860)	Single-file UI, native API-key input, ships with the model stack
Distribution	Docker image on ghcr.io + HF Spaces demo	Free unlimited pulls for public images; browser demo needs no install
Evaluation	500-question harness, 10 question types	Both pipelines run per question; results logged to SQLite

The two pipelines compared

Stage	Standard RAG	Hybrid RAG
Query analysis	None	spaCy NER extracts entities; cross-domain detection
Routing	Search all partitions	Partition routing weighted by entity frequency
Lexical retrieval	FTS5 BM25 top-k	FTS5 BM25 top-k (same engine)
Graph retrieval	None	Entity-graph traversal over 34M chunk-entity rows
Fusion	Single ranking	RRF fuses lexical and graph rankings
Selection	Fixed top-k	Confidence-calibrated threshold; adaptive chunk count
Escalation	None	Tier 4: cross-encoder reranking if confidence is low
Generation	LLM with citations	Same LLM, same citation format
Typical latency	~8 s	~17-30 s (CPU container)

Both pipelines end at the same generator with the same citation rules, so any quality difference comes from which chunks they hand it. That was a deliberate evaluation design decision: isolate the retrieval variable.

2. The Pipeline Stage by Stage (Worked Example)

This walkthrough traces one real query end to end. The query was run in a production Docker container against the full 16.9 GB database:

```
"What follow-up actions were planned after the Proxima Bank 429 incident?"
```

1 **Query received.** The Gradio web UI accepts the query plus the user's OpenAI API key. The key is passed per request to the generation layer and never stored. The engine is already warm: in lazy mode only the small models (NER, embeddings, cross-encoder) live in RAM; all 1.9M chunks stay on disk in SQLite.

2 **Entity extraction.** spaCy's statistical named-entity-recognition model tags 'Proxima Bank' as an organization and surfaces incident-related terms. Entities are looked up in the entity_frequency table (frequency per entity across the corpus).

3 **Cross-domain detection.** 'Proxima Bank' appears in multiple partitions (GitHub, Fireflies, HubSpot, Gmail), so the query is marked cross-domain. Cross-domain queries raise the retrieval threshold - they are exactly the questions a single-source keyword search tends to miss.

4 **Partition routing.** The system weights which of the 9 partitions to search, based on where the query's entities appear. This query's entities are most frequent in GitHub, so GitHub chunks get boosted during retrieval.

5 **Lexical retrieval (FTS5).** The query terms are matched against the chunks_fts virtual table. SQLite's BM25 implementation scores every chunk using Porter-stemmed terms ('planned', 'plans' and 'planning' all match 'plan') and returns a ranked candidate list, capped to keep query cost flat regardless of corpus size.

6 **Entity-graph retrieval.** In parallel, the extracted entities are looked up in chunk_entities (34M rows, indexed on entity and on entity+partition). Every chunk linked to 'Proxima Bank' becomes a graph candidate - including chunks that never contain the query's literal words but discuss the same entity.

7 **Reciprocal Rank Fusion.** The two rankings are merged with RRF: each chunk earns 1/(k + rank) from each list that contains it (k = 60), and the scores are summed. A chunk ranked high by both BM25 and the entity graph outranks a chunk that only one method likes.

8 **Confidence calibration and adaptive selection.** Fused scores are mapped to a calibrated confidence. An adaptive threshold (higher for cross-domain queries) decides how many chunks are enough - typically 3-5 - instead of a fixed top-k.

9 **Tier 4: last-resort reranking (conditional).** If calibrated confidence is still below the escalation threshold, only this query's own top candidates are escalated to the cross-encoder. It is a rare fallback, not a per-query default - that is what keeps hybrid latency acceptable over 1.9M chunks on CPU.

10 **Context assembly.** Selected chunks are concatenated with source citations (chunk id, original file path, partition). Token count is estimated from the context before generation so cost is known even before the API call.

11 **Generation.** The LLM is instructed to answer only from the provided evidence and to cite every claim as [chunk_id | source]. If the evidence does not contain the answer, it must say so - which is how the system avoids hallucination.

12 **Post-processing and persistence.** Relevance, confidence, latency and token counts are computed and, in evaluation mode, written to the comparisons table row by row as the batch runs.

What actually happened (both pipelines, live test)

Pipeline	Retrieved	Latency	Tokens	Outcome
Standard	3 chunks: HubSpot + 2 Fireflies meeting-note chunks	7.3 s	725	Correct: found the incident meeting notes, named all four owners and due dates
Hybrid	3 chunks: all GitHub (TSF weights, triage canaries)	53.3 s	693	Wrong partition: honest 'cannot determine' answer; 4.4% token savings

This single example shows the system's core trade-off in one frame. Standard won because the query's phrasing matched the meeting notes verbatim, which is BM25's home ground. Hybrid's entity routing pulled it toward GitHub, where the entities are most frequent - a real failure mode (Section 8). Both answers were honest: Hybrid's LLM reported that its evidence did not contain the incident actions rather than inventing them.

3. Techniques and Terms, Defined

Every technique the pipeline uses, in the order a reader meets them. These definitions are the vocabulary for the interview answers later in this document.

Term	Definition
RAG (Retrieval-Augmented Generation)	Answer architecture that retrieves relevant documents first and passes them as context to an LLM, instead of relying on the model's memorized knowledge. Grounds answers in real data and makes them citable.
Chunking	Splitting source documents into retrievable units (chunks) small enough to fit many into the LLM context window but large enough to carry meaning. This corpus: 1.92M chunks with per-chunk token counts stored in the database.
Token	The unit LLMs read and bill: roughly 0.75 words of English. Fewer context tokens means cheaper, faster, more focused answers - the metric the hybrid pipeline optimizes.
Lexical (keyword) search	Matching the literal terms of the query against document terms (TF-IDF, BM25, FTS5). Strong when wording matches; blind to synonyms and paraphrase.
TF-IDF	Classic lexical scoring: term frequency in a document times inverse document frequency across the corpus. Rare, topic-specific words count for more.
BM25	The standard modern lexical ranking function and the successor to TF-IDF. Adds term-frequency saturation (the 10th occurrence of a word adds less than the 1st) and document-length normalization (a long document does not win just by being long). Tuned by constants k1 and b.
FTS5	SQLite's full-text-search extension. Maintains an inverted index (term to list of documents containing it) so BM25 search over 1.9M chunks returns in milliseconds, using the MATCH operator and the bm25() rank function.
Inverted index	The data structure behind all full-text search: maps each term to the documents containing it, so a query reads a few short lists instead of scanning every document.
Porter stemmer	Algorithm that strips word endings ('planned', 'planning', 'plans' all reduce to 'plan') so lexical search matches morphological variants. Enabled in this system via the 'porter unicode61' tokenizer.
NER (Named Entity Recognition)	NLP task that tags real-world entities in text (people, organizations, dates). Here: spaCy's en_core_web_sm model, a statistical model that runs in milliseconds on CPU.
Entity graph / knowledge graph	The chunk_entities table: 34M (chunk, entity) links saying 'this chunk mentions this entity.' Enables retrieval by entity identity rather than by word matching - finding a chunk that says 'she shipped the fix' because the graph knows 'she' co-occurs with Anika Rao.
Partition routing	Restricting or weighting retrieval to the data sources (partitions) where the query's entities actually appear, instead of searching all 9 sources equally.
Dense embeddings	Text encoded by a neural model (here all-MiniLM-L6-v2, a 22M-parameter encoder) into vectors where similar meanings sit close together, measured by cosine similarity. Captures paraphrase that lexical search misses.
Cross-encoder reranking	A model that reads the query and a candidate chunk together and scores exact relevance. Far more accurate than first-stage retrieval but must run per candidate, so it is only affordable on a shortlist. Used here as a last-resort tier.
Bi-encoder vs cross-encoder	A bi-encoder encodes query and document separately (cheap, precomputable, slightly less accurate); a cross-encoder encodes them jointly (accurate, expensive). Standard retrieval pipelines use bi-encoders first and cross-encoders to rerank.
RRF (Reciprocal Rank Fusion)	Rank-based merge of multiple rankings: each list contributes 1/(k + rank) per item (k = 60 is standard). Merges BM25 and entity-graph lists without needing comparable score scales.
Hybrid search	Any retrieval combining lexical and semantic (embedding- or graph-based) signals; the two methods' errors are largely uncorrelated, so the union beats either alone.

Term	Definition
Confidence calibration	Mapping a raw retrieval score to a probability-like confidence, so a single threshold means the same thing across queries and question types.
Router (query router)	A cheap pre-pass that predicts whether the expensive pipeline is needed. Here: probe Standard's confidence first; strong confidence routes to Standard, weak or cross-domain signals route to Hybrid.
Hallucination	An LLM generating plausible but unsupported claims. Countered here by prompting for evidence-only answers with mandatory citations, so unsupported queries return 'cannot determine' instead of invented facts.

4. The Database: One 16.9 GB File

Everything the system knows lives in a single SQLite database, hybrid_rag_v22.db. No server, no credentials, no separate search engine - which is exactly why it can ship inside a Docker image that anyone can run. The schema as deployed:

Table	Rows	Purpose
documents	one per source file	Provenance: file path, partition, processed date, content hash (dedup)
chunks	1,924,473	The corpus: text, token count, entity count, chunk index, nullable embedding column
chunks_fts (FTS5)	mirrors chunks	Inverted index over chunk text, porter unicode61 tokenizer; BM25 scoring
chunk_entities	33,914,089	The knowledge graph: (chunk_id, entity) links with 4 indexes covering entity, partition, and combinations
entity_frequency	one per entity	Corpus-wide entity counts; powers stopword-style filtering of over-common entities
comparisons	one per pipeline per query	Evaluation log: answer, latency, tokens, relevance, confidence
usage	one per LLM call	Per-provider, per-model input/output tokens and USD cost
credit_accounts	one per provider	Budget ceilings per provider for cost control

Where the 16.9 GB goes, and the embedding surprise

The size is all load-bearing: roughly one third chunk text, one third the FTS5 inverted index (an index typically costs 2-3x the text it covers), and one third the 34M entity rows plus their four indexes. A diagnostic script revealed one instructive detail: the embedding column exists in the schema but is empty in practice - the SQLite path never reads stored embeddings back, because embeddings are computed in memory at runtime from chunk text. The column is dead weight by design choice; a planned cleanup pass confirms it before any size optimization. That diagnosis (SELECT COUNT on a 1.9M-row table) is a good interview story about measuring before optimizing.

Why SQLite, and when it would stop being enough

- **Zero operational surface.** One file: no server process, no auth, no connection pool. The whole system is one docker run command.
- **FTS5 is a real BM25 engine.** Millisecond ranked search over 1.9M chunks without standing up Elasticsearch.
- **The workload is read-heavy.** Ingestion happens once; queries only read (plus small comparison-log writes). That is SQLite's best case.

- **Code already supports the upgrade path.** The persistence layer has a PostgreSQL + pgvector mode with an HNSW vector index; SQLite was chosen for the shipped artifact, not because it is the only option.

The known limits: a single writer (fine here - writes are logs, not corpus edits), no horizontal scaling across machines, and a 16.9 GB file that gets awkward to move around 100 GB. Section 9 covers the migration plan.

5. The Smart Router and the Calibration Bug

The router is the system's reasoning loop: it spends a cheap probe to decide whether to pay for the expensive path. Standard RAG's confidence is measured first; above a high threshold the query is answered cheaply, below a low threshold it escalates to Hybrid, and in between a blended score of (1 - confidence), query complexity, and cross-domain evidence decides.

The bug that made the router meaningless

The original calibration compressed every query's probe confidence into a narrow band around 0.895 - regardless of whether retrieval had actually found good evidence. Every query therefore landed in the blended middle zone, and the router's 'decisions' were noise around a constant. The signal that exposed it was distributional: a working calibrator should spread confidences across the unit interval; a constant 0.895 for all 500 questions means the function is degenerate. The fix adjusted the calibration pivot so scores distribute properly, and the router began making genuinely different decisions per query (post-fix tests showed 56-59% confidence queries staying Standard and weaker or cross-domain ones escalating).

The product decision: force Hybrid by default

After calibration, the shipped app still runs Hybrid on every query by setting the router's high-confidence threshold out of reach. The reasoning: for enterprise search, a human would otherwise spend minutes finding these answers manually, so ~10 extra seconds per query is cheap; the token savings (7.7%) and relevance gain (+0.9 pp) apply to every query; and predictable quality beats adaptive quality for a demo. The router remains in the codebase as a configuration option - one environment variable restores it for latency-sensitive deployments. Being able to articulate when the router should win (interactive chat, autocomplete, high QPS) versus when forced Hybrid should win (batch research, quality-critical search) is the actual interview point.

6. The Evaluation Harness

The harness (batch_compare_v2.py) runs 500 questions spanning 10 question types from single-source lookups to cross-domain synthesis, drawn from a question bank stored as Parquet. For every question it executes both pipelines unconditionally, logs results to the comparisons table after each question (so a crashed run loses one question, not the batch), and tracks per-provider token spend and USD cost in the usage table.

Metrics and how each is defined

Metric	Definition	Success criterion
Answer relevance	Score of the generated answer against the question's expected evidence	Hybrid at least matches Standard
Tokens used	Estimated context tokens handed to the LLM	Net reduction (cost proxy)
Latency	time.perf_counter around the whole pipeline	Acceptable increase for the quality gained
Win rate	Pairwise comparison of pipeline answers per question	Hybrid wins more questions than it loses

Results (500 questions)

Metric	Standard	Hybrid	Difference
Answer relevance	65.3%	66.1%	+0.9 pp
Total tokens	459,014	423,874	-7.7%
Win rate	36%	39%	+3 pp
Typical latency	~8 s	~17-30 s	+~10 s

Two findings shaped the product more than the headline numbers. First, the per-question distribution: Hybrid used fewer tokens than Standard on 39% of questions but more on 31% - the net 7.7% saving comes from the fusion doing its job on hard questions while occasionally over-retrieving on easy ones. Averaged metrics hide this; the harness's per-question rows exposed it. Second, the harness is what surfaced the router calibration bug (Section 5): every question showing identical 0.895 probe confidence is not a coincidence, it is a degenerate function. That is the cleanest example of evaluation driving a real system fix.

7. Memory and State Design

The central decision: a 16.9 GB corpus must never need to fit in RAM. The system runs in lazy, disk-backed mode - chunk text and the entity graph stay in SQLite and are touched only through indexed queries per retrieval. What does live in memory is small and fixed: the NER model, the embedding model, and the cross-encoder (hundreds of MB combined). The database's page cache does the rest.

What is held where

State	Lives in	Lifetime
1.92M chunk texts + FTS index	SQLite on disk, queried per retrieval	Permanent (the shipped artifact)
34M entity-graph rows	SQLite on disk, indexed lookups	Permanent
NER / embedding / cross-encoder models	RAM, loaded once at startup	Process lifetime
Query embeddings	Computed in memory at query time, never persisted	Per query
Router decision	Computed, logged, discarded	Per query
Evaluation results	comparisons table, written per question	Permanent (queryable history)
Token spend and USD cost	usage + credit_accounts tables	Permanent

Two design points worth volunteering in an interview. Embeddings are deliberately not persisted in the SQLite build: the retrieval path that would read them back is not part of the primary pipeline, and storing them as JSON text would roughly double the database for no query-time benefit (this was verified in the code before acting on it). And there is no conversational state: each query is self-contained, which makes the service stateless and trivially restartable - the natural extension, session memory, is discussed in Section 12's interview answers.

8. Failure Modes and Mitigations

Every failure below was observed in live testing, not hypothesized. Each one is a better interview story than a success, because each comes with a diagnosis.

Failure mode	Observed as	Root cause	Mitigation
Degenerate router calibration	0.895 probe confidence on all 500 questions	Calibration function compressed all scores into one band	Re-pivot the calibration; verify on the eval distribution, not spot checks
Entity-frequency dominance	Anika Rao query: Hybrid pulled 3 GitHub chunks while the answer sat in Fireflies meeting notes	Routing follows where entities are most frequent, not where the answer is	Known limit; candidate fixes: source-diversity penalty in fusion, learned router, or reciprocal partition scoring
Verbatim phrases beat entity routing	Nimbus Analytics query: Standard found the Gmail chunk; Hybrid missed with 84.6% confidence	BM25 matches exact phrasing; entity graph adds nothing when wording matches perfectly	Fusion already keeps the lexical tier; the loss is inherent to when graph signals dominate
Confidence inversion	Hybrid 84.6% confident on wrong retrieval; Standard 59.7% on the correct one	Confidence measures answer-evidence fit, not retrieval correctness	Never gate on confidence alone; always surface sources for human verification
Rate-limit backoff doubling latency	Same query: 26.0 s alone, 50.4 s in side-by-side mode, identical chunks and tokens	OpenAI SDK silently retries throttled requests with exponential backoff	Diagnose via logs before blaming code; space calls or raise tier; document that single-query latencies are the honest numbers
Tier-4 reranking cost on CPU	53.3 s hybrid latency on one query	Cross-encoder scoring many candidates on CPU, in a throttled container	Cap escalation candidates; GPU variant available but not shipped (Section 10)

One non-failure to know cold: when the evidence genuinely lacks the answer, the system returns 'cannot determine from the provided evidence' with what the evidence does show. In live tests of a question whose answer was absent from the corpus (GitHub PRs for a specific incident), both pipelines refused to invent - which is the anti-hallucination design working. An interviewer may probe exactly this difference between a miss and a lie.

9. Scaling: What Breaks Next

The system is comfortably within a single machine's envelope today: 1.9M chunks, 16.9 GB, read-mostly workload, millisecond FTS5 and indexed entity lookups. Honest scaling analysis means naming the walls in order.

1. **Distribution (already hit, already solved).** The artifact is the bottleneck, not the engine: the Docker image is ~17 GB because the corpus travels inside it. Mitigations shipped: compression during transfer (text halves in transit), a free ghcr.io registry with unlimited public pulls, and a 3-partition browser demo for try-it users.
2. **Concurrent writers.** SQLite allows one writer. Fine now (writes are evaluation logs); breaks if ingestion becomes continuous. Wall at roughly the point where sources update hourly.
3. **Corpus growth.** Around 100 GB the single file becomes awkward to move and back up, and index rebuilds stretch into hours. That is the natural trigger for the PostgreSQL migration the code already supports (partitioned tables, pgvector HNSW for the vector tier, read replicas).
4. **Vector tier at scale.** If embeddings ever became a first-class retrieval path, brute-force cosine similarity over 1.9M vectors is seconds per query; pgvector's HNSW approximate index brings that back to milliseconds. The code's Postgres mode exists precisely for this branch.
5. **LLM rate limits.** The wall actually hit first in practice: back-to-back pipeline comparisons triggered provider throttling (Section 8). Horizontal query scaling needs request spacing, higher tiers, or multiple keys - a business constraint, not an architecture one.

6 **Cross-partition queries at 10x sources.** Entity-frequency routing degrades as partitions multiply (the Anika Rao failure generalizes). The principled fix is a learned router trained on the evaluation harness's labeled outcomes.

10. Why Each Tool Was Chosen (and What Was Rejected)

Decision	Chosen	Rejected	The reasoning
Storage	SQLite + FTS5	PostgreSQL/pgvector, Pinecone, Weaviate	The shipped artifact must run anywhere with zero setup; FTS5 gives real BM25 without another server. The Postgres mode is implemented for the scale path; a managed vector DB is operational overhead the corpus does not need yet
Entity extraction	spaCy NER	LLM-based extraction	NER runs per query in milliseconds on CPU at zero marginal cost; an LLM extractor adds latency and cost to every query for marginal recall
Generation	OpenAI GPT	Self-hosted Llama	Strong instruction-following for citation-only answers; per-user API keys make the public demo free to operate. Self-hosting would shift GPU cost onto the deployment
Reranking	MiniLM cross-encoder, last-resort tier	Always-on reranking, larger rerankers	Accuracy gain is real but per-candidate cost is high; gating on low confidence keeps the 95% case fast
Image runtime	CPU-only PyTorch	CUDA build	GPU adds ~5 GB of NVIDIA libraries to pull for every user, most of whom have no GPU; the bottleneck is the LLM API call, not local compute (verified: only tier-4 reranking benefits)
UI	Gradio	Streamlit, Flask+React	Single-file UI with native password fields and API-key flow; zero build step
Distribution	Docker + ghcr.io; HF Spaces demo	Setup script, email-the-file	One command for users, exact dependencies guaranteed; free unlimited pulls for public images. The browser demo covers everyone who will not download 17 GB

11. The Five Interview Questions, Answered

Model answers in first person. Each is calibrated to about 90 seconds spoken; the 'if pressed' lines handle follow-ups.

Q1. Describe one agent or AI system you architected end-to-end.

I built an enterprise hybrid RAG search system over nine workplace data sources - 1.9M document chunks and a 34-million-row entity graph in a single 16.9 GB SQLite database. The reasoning loop is a query router: a cheap confidence probe decides between a lexical pipeline and an entity-aware hybrid pipeline, which itself escalates through tiers - BM25 retrieval, entity-graph fusion, and cross-encoder reranking only when calibrated confidence is low. Tool use is the retrieval stack itself: FTS5 full-text search, entity-graph lookups, RRF fusion, and an LLM generator under citation-only prompting. Memory is deliberately disk-backed: the corpus is never loaded into RAM; only the small NER, embedding, and cross-encoder models are resident. Evaluation was a 500-question harness running both pipelines per question, which is also how I found and fixed a degenerate router calibration bug. In production it ships as a 17 GB Docker image with a web UI, plus a browser-based demo; live testing surfaced concrete failure modes - entity-frequency dominance, confidence inversion, provider rate-limit backoff - each diagnosed from logged evidence. One framing I volunteer up front: this is agentic routing over a retrieval pipeline, not an autonomous multi-step agent; the loop is per-query and my design kept it that way deliberately, because enterprise search wants predictable latency over autonomous exploration.

If pressed: where would true agent behavior come in? Multi-hop research queries (query, read, reformulate, search again) and tool-selection loops over the partitions - the router architecture extends naturally.

Q2. Which agent frameworks or orchestration stacks have you used in production?

None of the agent frameworks - LangChain, LlamaIndex, CrewAI - in this system, and that was a deliberate choice, not a gap. The pipeline is custom Python: about 2,600 lines where every retrieval decision, threshold, and prompt is mine to instrument. The trade-off I accepted: I wrote my own chunking, routing, fusion, and evaluation instead of composing them, and in return every latency number and token count in the evaluation is attributable to code I can point at - which is exactly what made the router bug findable. What I do run in production: Gradio for the serving layer, Docker and ghcr.io for distribution, Hugging Face tooling for models and the demo Space, and SQLite/FTS5 as the data plane. My honest position: frameworks are the right choice for velocity when the pipeline is a means to an end; they are the wrong choice when the pipeline itself is the product being evaluated.

If pressed: I would reach for LangChain's retrieval primitives to prototype an ingestion variant, then port the winner into the custom pipeline once measured.

Q3. Describe an evaluation or harness system you built for AI workflows.

A 500-question, 10-question-type harness over the enterprise corpus, stored as a Parquet question bank, executed by a batch runner that runs both pipelines unconditionally per question and writes each result to SQLite as it goes - a crash costs one question, not the batch. It measured four things: answer relevance against expected evidence, context tokens (the cost proxy), wall-clock latency, and a pairwise win rate between pipelines. Success was defined before results existed: Hybrid wins if it matches Standard's relevance while cutting tokens, with latency increase kept under a stated budget. It found the net 7.7% token reduction and +3-point win rate - and, more importantly, two things averages hid: Hybrid uses more tokens than Standard on 31% of individual questions, and every question in one early run showed identical 0.895 router confidence, which is how I knew the calibration was degenerate rather than the router being genuinely confident. The harness also tracked per-provider USD cost in a usage table, so quality claims always shipped attached to a cost number.

Q4. What were the most important failure modes, and how did you mitigate them?

Three, in order of what they taught. First, the degenerate router calibration: every query scored 0.895, so the router was decorative. Found via distribution analysis in the harness, fixed by re-pivoting the calibration; the lesson is that calibrated scores must be validated on their spread, not spot-checked. Second, entity-frequency dominance: a person-entity query routed Hybrid to GitHub, where the entity is most frequent, while the answer lived in meeting transcripts - the router follows frequency, not answer location. Mitigation is honest: it is a known limit with a principled fix (source-diverse fusion or a learned router), and side-by-side mode exists precisely to expose such cases. Third, confidence inversion: Hybrid reported 84.6% confidence on the wrong retrieval while Standard's correct answer scored 59.7% - because confidence measures answer-evidence fit, not retrieval correctness. The mitigation is design policy: never gate user-facing trust on confidence alone; always show sources. Alongside those, operational failure modes: provider rate-limit backoff silently doubling latency (diagnosed by identical token counts with doubled wall time), and CPU throttling in containers.

Q5. How did you handle memory and state for long-running tasks?

By refusing to hold the corpus in memory. The system runs disk-backed lazy retrieval: 1.9M chunks and the 34M-row entity graph stay in SQLite and are reached only through indexed queries, so memory footprint is the three small models, not the data - and the same binary handles a corpus 10x larger by adding disk, not RAM. State that must survive is written transactionally per unit of work: each evaluation question commits its result row immediately, so a multi-hour batch evaluated across 500 questions is restartable without losing more than the current question. Long-running cost state - per-provider tokens and USD - accumulates in usage and credit-account tables, which is how budget ceilings are enforced. Deliberate non-feature: no conversational memory across queries; each query is stateless, which keeps the service horizontally restartable. The natural extension I can design out loud: session-scoped state as a cache keyed by user and conversation, with retrieved context summarized and capped - the same evidence-budget discipline the pipeline already applies within a query.

12. Numbers to Know Cold, and Rapid-Fire Questions

Number	Value
Chunks in corpus	1,924,473
Entity-graph rows	33,914,089 (about 34M)
Data sources (partitions)	9: Slack, Gmail, GitHub, Jira, Confluence, HubSpot, Linear, Fireflies, Google Drive
Database size	16.9 GB, single SQLite file
Evaluation questions	500, across 10 question types
Token usage	459,014 (Standard) vs 423,874 (Hybrid): -7.7%
Answer relevance	65.3% vs 66.1%: +0.9 pp
Win rate	36% vs 39%: +3 pp
Latency	~8 s Standard; ~17-30 s Hybrid in a CPU container
Per-question token behavior	Hybrid cheaper on 39% of questions, costlier on 31%
Router thresholds	low confidence 0.42; blended-score threshold 0.50; high-confidence out of reach in forced-Hybrid mode
Models	spaCy en_core_web_sm (NER); all-MiniLM-L6-v2 (embeddings, 22M params); MiniLM cross-encoder (reranking)

Rapid-fire follow-ups, one-line answers

- **Why RRF and not score averaging?** BM25 and graph scores live on different scales; RRF needs only ranks, so no per-method normalization tuning.
- **Why Porter stemming?** Morphological variants ('planned'/'planning') match without maintaining a synonym list.
- **How do you stop hallucination?** Citation-only prompting plus an explicit escape hatch: answer from evidence or say it cannot be determined.
- **Why is Hybrid sometimes slower than the LLM call itself?** Tier-4 cross-encoder reranking on CPU; it is gated to low-confidence queries precisely for this reason.
- **What would you do first with more time?** A learned router trained on the harness's 500 labeled outcomes; it directly attacks the entity-frequency failure mode.

- **Biggest surprise?** That the embedding column was dead weight - written but never read - and that finding it took one COUNT query after shipping.
- **Why does the demo force Hybrid?** Quality and token savings on every query beat latency for enterprise search; the router stays one config flag away for latency-sensitive products.
- **How do you know the router works now?** Confidence distributes across queries and makes different decisions; two live queries with 56-59% probe confidence stayed Standard while weak-evidence ones escalated.
- **What breaks first at 100 GB corpus?** The single-file artifact's movability and index rebuild times - that is the Postgres migration trigger, and the code path already exists.
- **Why show users the sources?** Because confidence measures evidence fit, not truth - the human check is part of the design (see the 84.6% inversion case).